Disciplina: Sistemas Distribuídos
Professora: Ana Cristina Barreiras Kochem Vendramin

Avaliação (valor 2,5)
Replicação de dados. Eleição. Tolerância a Falhas. gRPC.

Implementar um sistema distribuído de replicação de dados, conforme ilustrado na Figura 1, utilizando o modelo **Pull (*Fetch-based*, estilo KRaft/Kafka)**, **eleição de líder (estilo Raft)** e suporte a **leitura eventual em qualquer nó**.

O ambiente é composto por **5 processos distintos**:

- **1 Cliente**
- **4 nós do *Cluster* (N = 4).** Estes nós possuem o mesmo código e transitam dinamicamente entre os papéis: Seguidor, Candidato e Líder.

A comunicação entre todos os nós deve ser realizada exclusivamente via **gRPC**.

Conceitos e Regras de Funcionamento

Estrutura do *Log*, Termo e Eleição

- **Índice do *log*:** posição sequencial de um registro no *log*;
- **Termo (*term*):** representa o mandato da liderança atual.
  - Incremento do Termo: quando um nó não recebe comunicação do líder por um tempo (*election timeout*), ele se torna Candidato e incrementa o seu termo: Termo = Termo + 1. O novo Líder eleito passa a registrar todas as novas entradas de *log* com esse número de Termo.
  - Atualização Obrigatória de Termo: se qualquer nó (incluindo o líder) receber uma mensagem ou resposta contendo um Termo estritamente maior que o seu termo local atual, ele deve imediatamente atualizar seu termo para esse novo valor e se transformar em Seguidor.
  - O novo Líder eleito passa a registrar todas as novas entradas de *log* com esse número de Termo.
- **Voto Único:** um nó só pode votar em **no máximo um candidato por Termo**. Um nó só concede seu voto a um candidato se o *log* do candidato for tão ou mais atualizado que o seu próprio *log*.

Quórum, Modelo *Pull* e Detecção de Falhas

- **Quórum fixo:** o quórum para decisões no *cluster* é a maioria absoluta dos nós. Para este *cluster* de 4 nós, o quórum é de **3 nós**.

- **Sem quórum, sem escrita:** se o líder não conseguir confirmar a replicação no quórum (pelo menos 3 nós, incluindo ele mesmo), a operação **não pode ser comitada** e a escrita do cliente fica bloqueada/rejeitada.
- ***FetchRequest* como *Heartbeat*:** as réplicas enviam requisições periódicas de *FetchRequest* ao líder (ex: a cada 100ms) informando seu último termo e último índice. Essa requisição serve para cada réplica **buscar novos dados**, **confirmar o recebimento de *logs*** e avisar que está viva (***heartbeat***).
- **Temporizador do líder:** o líder mantém um *Heartbeat Timeout* individual para cada réplica. Se uma réplica parar de enviar *FetchRequests*, o líder a considera inativa.
- **Monitoramento de inatividade das réplicas pelo líder**: o líder registra o instante do último *FetchRequest* recebido de cada réplica. Se o tempo decorrido ultrapassar o *Heartbeat Timeout*, a réplica é marcada como inativa. Isso permite que o Líder identifique imediatamente uma possível perda de quórum e recuse/bloqueie novas escritas do cliente, em vez de aceitar dados que ele sabe que não poderão ser comitados.
- **Índice de *commit*:** o maior índice do *log* que já foi replicado e armazenado na maioria dos nós (quórum). Dados com índice do *log* ≤ índice do *commit* são considerados comitados/efetivados e aplicados no banco de dados final.

Pontuação:

Cliente (0,30)

- **(0,15) Gravação com Redirecionamento:** envia solicitações de escrita ao líder. Se tentar gravar em um seguidor ou candidato, ele receberá a indicação de quem é o líder atual.
- **(0,15) Consulta nos nós do *cluster*:** permite realizar leituras de dados em qualquer um dos nós do cluster.

Detecção de Falhas e Eleição de Líder (0,60)

- **(0,25) Temporizador aleatório e transição de Papéis:** detecção da queda do líder via ausência de comunicação. Transição correta: seguidor → Candidato → líder. Cada réplica possuirá um temporizador aleatório para detectar falhas no líder, reiniciado a cada *FetchRequest* bem-sucedido. Dessa forma, cada réplica ativará o papel de candidato em momentos distintos, evitando a divisão de votos (*split vote*).
- **(0,25) Votação (*RequestVote*):** execução da eleição garantindo incremento de termo e a regra de voto único por termo.
- **(0,10) Validação de *Log* para Voto:** restrição de voto baseada na atualização do *log* do candidato.

Líder e Replicação *Pull* (0,80)

- **(0,20) Recebimento de gravação e *log* local:** atribui o termo atual e o índice sequencial para novas gravações do cliente.
- **(0,20) Resposta de *FetchRequests* e Controle de Quórum:** atende aos *FetchRequests*. Monitora a atividade das réplicas via temporizadores e avança o índice de *commit* apenas ao obter confirmação do quórum, a qual é calculada a partir da análise dos índices de log informados por cada réplica nas requisições de *FetchRequest*. A réplica efetiva (*commit*) o dado no seu banco local apenas quando enviar um novo *FetchRequest*, através do qual receberá o índice de *commit* atualizado do líder.
- **(0,20) Confirmação de *Commit* ao cliente:** confirma o sucesso ao cliente somente após o avanço do índice de *commit*.
- **(0,20) Persistência em Disco:** persiste em disco o termo, o voto concedido, o *log* de operações e os dados comitados.

Réplicas e Resolução de Inconsistência (0,80)

- **(0,20) *Loop* de Fetch Ativo:** busca ativamente entradas no líder atual e aplica o índice de *commit* recebido para efetivar os dados no banco local, tornando-os visíveis para leitura.
- **(0,30) Truncamento e Correção de *Log*:** ao detectar divergências de *log*, o nó trunca o *log* local a partir do ponto conflitante e ressincroniza com o líder atual.
- **(0,15) Leitura Eventual Consistente:** responde a consultas do cliente utilizando apenas dados comitados.
- **(0,15) Persistência Local:** garante a gravação em disco do estado do nó para resistir a desligamentos.

Cenários de Testes Obrigatórios para a Apresentação

1. **Escrita Normal e Leitura Distribuída:** gravar um dado no líder e demonstrar que a leitura se torna disponível em todas as réplicas.
2. **Eleição Automática por Queda do Líder:** derrubar o líder e demonstrar a eleição automática de um novo líder com termo incrementado.
3. **Bloqueio por Perda de Quórum:** derrubar réplicas até N < quórum (menos de 3 nós ativos) e demonstrar que o líder recusa/bloqueia novas escritas do cliente.
4. **Recuperação e Truncamento de Log** (Réplica Desatualizada ou Queda do Líder):
   - Cenário A (Réplica Desatualizada): desligar um seguidor, realizar novas escritas com sucesso no *cluster* e, ao religá-lo, demonstrar que ele busca os dados faltantes via *FetchRequest*.
   - Cenário B (Truncamento por Divergência): enviar uma escrita para o líder (N_1), fazê-lo gravar localmente, mas derrubá-lo imediatamente (simulando *crash*) antes da replicação. Aguardar a eleição de um novo líder (N_2, em um novo termo), realizar uma nova escrita com sucesso e, ao religar N_1, demonstrar que ele muda para Seguidor, trunca (deleta) a entrada não comitada e ressincroniza o *log* correto via *FetchRequest*.

5. **Isolamento de Dados *Uncommitted*:** provar que dados recebidos via *FetchRequest* permanecem invisíveis para leitura até que o índice do *commit* autorize o armazenamento no banco final.

Persistência Obrigatória

Todos os nós do *cluster* (líder e réplicas) devem persistir:

1. O **Termo atual** e o **Voto concedido** (*voted_for*);
2. O ***log* de operações** (entradas com termo e índice);
3. O **estado do banco final** (dados comitados).

Requisitos Gerais e Interface

- Na execução da aplicação devem ser exibidos: o **papel do nó** (*seguidor/candidato/líder*), **termo atual**, **índice de *commit***.
- A defesa com explicação do código e execução dos cenários é **obrigatória** para a atribuição da nota.
- O desenvolvimento do sistema pode ser individual ou em dupla.

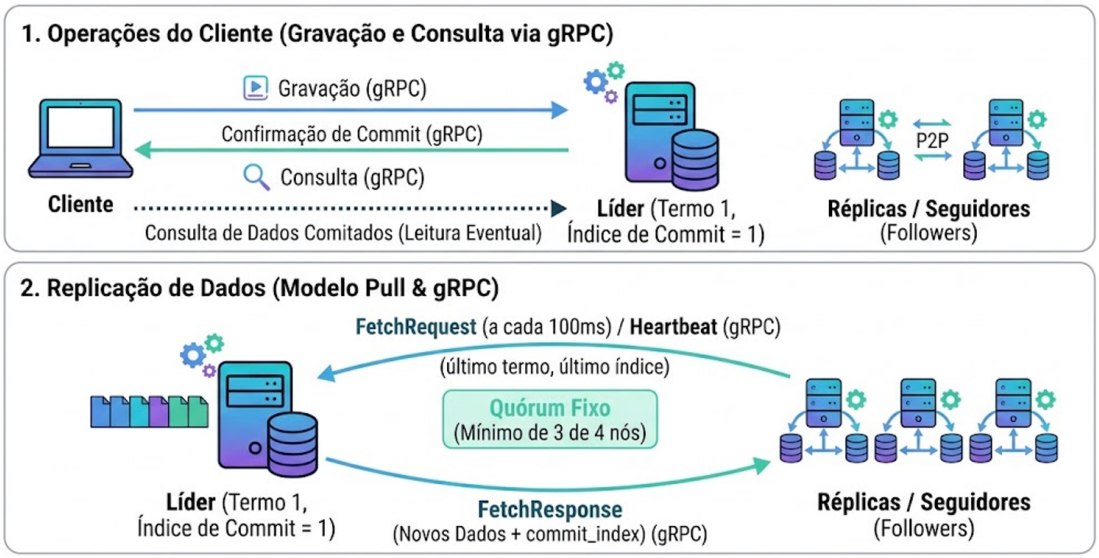


Figura 1 – Replicação de Dados via *Pull* com gRPC